

2D Sliding Puzzle — Project Plan

Playable sliding tile puzzle (3x3, 3x4, or 4x4) built step by step in Python + pygame, with a companion solver planned as a separate follow-on project.

Build order

Build in this order, and get each stage fully working (with prints, not graphics, where noted) before moving to the next one. That way when something breaks, it's clear which layer it's in.

Stage 0 — Setup

Install pygame, and confirm it works with the smallest possible program (blank window, filled with a color, stays open until closed). Get comfortable with the pygame event loop shape (pygame.init(), a while running: loop, pygame.event.get(), pygame.display.flip()) before writing any puzzle logic.

DONE — environment set up (Python 3.12.10, pygame 2.6.1, VS Code).

Stage 1 — Board representation (no graphics)

Decide how the board is stored: a list of lists, board[row][col], with 0 representing the blank. Write a function that builds the SOLVED board for a given size, and print it to the console in a readable grid. Get this right before anything else — every later stage depends on it.

NEXT UP

Stage 2 — Core movement logic

Write small, separately-testable functions: find_blank(board), legal_moves(board), apply_move(board, direction), and is_solved(board). Test entirely in the terminal with a text-based loop (print board, type a direction, apply it, print again) before touching graphics — this proves the logic is correct on its own.

PENDING

Stage 3 — Shuffling

Don't place tiles randomly (about half of all random arrangements are unsolvable). Instead start from the solved board and make a couple hundred random LEGAL moves, skipping the immediate reverse of the previous move. This guarantees a solvable, well-mixed board.

PENDING

Stage 4 — Rendering

Bring in pygame. Compute a pixel size per tile, and for each board[r][c] draw a rectangle at (col*tile_size, row*tile_size) with the number centered on it. Skip drawing the blank. Get a static shuffled board rendering correctly before wiring up input.

PENDING

Stage 5 — Input

Two input paths, both calling the Stage 2 functions (no duplicated logic): arrow keys mapped to a direction on KEYDOWN, and mouse clicks converted from pixel position to (row, col), moving only if adjacent to the

blank.

PENDING

Stage 6 — Game feel

Add a move counter and a timer (start on shuffle, stop when `is_solved()` becomes true), display a "Solved!" message, and add a key to reshuffle.

PENDING

Stage 7 — Multiple sizes

Generalize Stages 1-3 to take rows/cols as parameters rather than assuming 3x3, so 3x4 and 4x4 work by passing different arguments in.

PENDING

Common pitfalls to watch for

- Mixing up (row, col) vs (x, y) — screen x corresponds to column, y to row; this trips people up constantly.
- Forgetting that a "solvable" shuffle requires the random-legal-moves approach, not a fully random arrangement.
- Testing input handling before verifying Stage 2 works standalone — makes bugs much harder to isolate.